

UNIwersytet Kardynała Stefana Wyszyńskiego
w Warszawie

Wydział Matematyczno-Przyrodniczy
Szkoła Nauk Ścisłych

Weronika Plichta

Nr alb.: 115927

Kierunek studiów: informatyka

Porównanie metod rozpoznawania nowotworu mózgu przy użyciu modeli VGG, Inceptionv3 i sieci CNN

Praca licencjacka

Promotor:

dr inż. Robert A. Kłopotek

Warszawa 2023

Spis treści

Spis treści	2
Streszczenie	5
Spis rysunków	6
Spis tabel	8
1. Wstęp	11
2. Problem badawczy	13
2.1. Klasyfikacja obrazów	13
2.2. Konwolucyjna sieć neuronowa - CNN	13
2.2.1. Warstwa konwolucyjna	13
2.2.2. MaxPooling	14
2.2.3. Funkcje aktywacji	14
2.3. Transfer learning	15
2.4. Model VGG16	16
2.4.1. Zastosowanie TL	16
2.5. Model VGG19	16
2.6. Model Inceptionv3	17
3. Implementacja	19
3.1. Architektura programu	19
3.2. Wymagania systemowe	20
3.3. Opis klas i głównych metod	20
3.4. Przykłady uruchomienia	23
4. Metodologia badań	25
4.1. Opis zbioru danych - CE-MRI	25
4.2. Miary jakości klasyfikacji	26
5. Wyniki	29
5.1. Wyniki badań	29
5.1.1. Wyniki VGG16	29
5.1.2. Wyniki VGG19	30
5.1.3. Wyniki Inceptionv3	31
5.1.4. Wyniki własnej sieci CNN	32
5.1.5. Porównanie czasu treningu modeli	33

5.2. Analiza wyników	33
5.3. Dyskusja - porównanie wyników	34
6. Podsumowanie	35
Bibliografia	37

Streszczenie

Celem pracy jest porównanie metod klasyfikacji obrazów w kontekście klasyfikacji nowotworu mózgu na bazie danych CE-MRI składającej się z skanów rezonansu magnetycznego. Do tego celu wykorzystano modele VGG16, VGG19, Inceptionv3 oraz opracowano własną sieć konwolucyjną. W celu poprawy efektywności modeli, zastosowano transfer learning. W pracy zostały opisane algorytmy wykorzystywane podczas treningu sieci neuronowych, wykonanie programu klasyfikującego skany rezonansu magnetycznego i porównanie wyników.

Słowa kluczowe: konwolucyjne sieci neuronowe, transfer learning, VGG16, VGG19, Inceptionv3, klasyfikacja obrazów, klasyfikacja nowotworów

Abstract

The aim of this study is to compare image classification methods for brain tumor classification based on CE-MRI data consisting of magnetic resonance scans. The employed methods include VGG16, VGG19, Inceptionv3 models, and a custom convolutional neural network. Transfer learning was applied to enhance the effectiveness of the models. The research describes the algorithms used during neural network training, the implementation of the MRI scan classification program, and the comparison of the results.

Keywords: convolutional neural network, transfer learning, VGG16, VGG19, Inceptionv3, image classification, keras, brain tumor classification

Spis rysunków

2.1. Przykład warstwy konwolucyjnej (źródło [13])	14
2.2. Max Pooling (źródło [13])	14
2.3. Traditional/Classical ML vs. TL (źródło [9])	15
2.4. Architektura VGG16 i VGG19 (źródło [17])	16
2.5. Architektura modelu Inceptionv3 (źródło [15])	17
3.1. Architektura programu	20
4.1. Przykładowe skany rezonansu magnetycznego oponiaka	25
4.2. Standardowa macierz pomyłek (źródło [12])	26
4.3. Macierz pomyłek dla 3 klas (źródło [12])	27
5.1. Macierze pomyłek VGG16	29
5.2. Funkcja strat i dokładność przy 35 epokach modelu VGG16	30
5.3. Macierze pomyłek VGG19	30
5.4. Funkcja strat i dokładność przy 35 epokach modelu VGG19	31
5.5. Macierze pomyłek Inceptionv3	31
5.6. Funkcja strat i dokładność przy 35 epokach modelu Inceptionv3	32
5.7. Macierze pomyłek CNN	32
5.8. Funkcja strat i dokładność przy 35 epokach modelu CNN	33

Spis tabel

4.1. Szczegóły zbioru danych CE-MRI	25
5.1. Tabela jakości klasyfikacji dla VGG16	29
5.2. Tabela jakości klasyfikacji dla VGG19	30
5.3. Tabela jakości klasyfikacji dla Inceptionv3	31
5.4. Tabela jakości klasyfikacji dla CNN	32
5.5. Czas treningu i testowania poszczególnych modeli	33

Rozdział 1

Wstęp

Guzy mózgu, w tym glejaki, stanowią duże zagrożenie dla życia pacjentów, ponieważ ich przebieg kliniczny jest wysoko złośliwy [1]. Wczesna diagnoza i odpowiedni dobór terapii jest kluczowy do leczenia lub wydłużenia życia pacjenta. Współczesna diagnostyka skupia się na tomografii komputerowej, badaniom histopatologicznym i skanowaniu rezonansem magnetycznym (MR). Diagnoza i klasyfikacja wymaga specjalistycznej wiedzy doświadczonego eksperta.

Aby ułatwić ten proces, a tym razem zwiększyć dostępność diagnozy, w przeszłości zostało przeprowadzonych wiele badań nad klasyfikacją guzów mózgu na podstawie skanów rezonansu magnetycznego przy użyciu konwolucyjnych sieci neuronowych. W szczególności wykorzystanie wielowarstwowych modeli takie jak VGG16 zyskały popularność w tej dziedzinie, ze względu na ich dużą skuteczność w rozpoznawaniu obiektów oraz zdolność do wydobycia cech z obrazów. Dostosowanie tych modeli uzyskiwane jest za pomocą transfer learning [3].

Celem tej pracy jest porównanie modeli, który jest najefektywniejszy pod względem klasyfikacji guzów mózgu na podstawie skanów rezonansu magnetycznego.

Drugi rozdział to określenie problemu badawczego. Opisany jest problem klasyfikacji obrazów, transferu początkowych wartości wag i konwolucyjna sieć neuronowa. Przedstawione jest architektura wielowarstwowych głębokich modeli.

W trzecim rozdziale opisana została implementacja programu. Skupia się na wymaganiach systemowych i architekturze programu. Podane są przykłady uruchomienia programu z poziomu testowania modelu.

Kolejny rozdział zawiera metodologię badań, czyli opis zbioru danych, wykorzystane jakości klasyfikacji oraz konfiguracje parametrów modeli.

Ostatni rozdział to przedstawienie wyników. Przeprowadzono analizę i porównanie skuteczności wytrenowanych modeli.

Słowniczek

- CNN - (ang. Convolutional neural network) Konwolucyjna sieć neuronowa
- TL - (ang. Transfer learning)
- ML - (ang. Machine Learning) uczenie maszynowe

Rozdział 2

Problem badawczy

2.1. Klasyfikacja obrazów

Klasyfikacja obrazów z wykorzystaniem technik sztucznej inteligencji stała się popularna ze względu na rozwój złożonych sieci neuronowych, zwiększenie ogólnodostępnych baz danych, a przede wszystkim na wykorzystaniu jej w praktycznych zastosowaniach. [3] Praca skupia się na klasyfikacji obrazów w kontekście diagnozowania nowotworu mózgu na skanach rezonansu magnetycznego. Zaproponowany przeze mnie program klasyfikuje jeden z trzech typów nowotworu.

2.2. Konwolucyjna sieć neuronowa - CNN

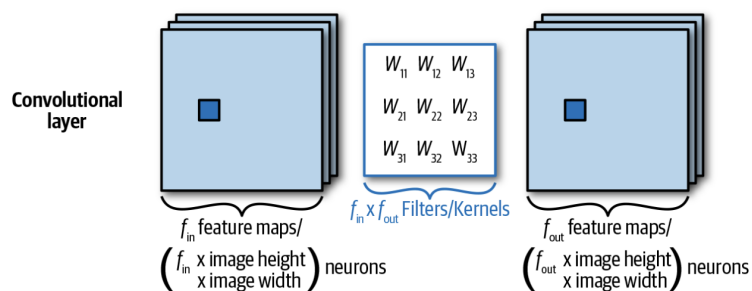
Konwolucyjne sieci neuronowe (CNN) są powszechnie stosowane do analizy obrazów. Podstawowe wersje składają się z warstw konwolucyjnych, poolingu i opcjonalnie funkcji aktywacji. Główną cechą jest wykorzystywanie warstw konwolucyjnych, które skupiają się na wykrywaniu lokalnych wzorców i cech w danych wejściowych. Po bloku (lub pojedynczej warstwie konwolucyjnej) następuje pooling. dane są przekształcane za pomocą jednej lub więcej warstw w pełni połączonych (fully connected layers), gdzie każdy neuron jest połączony z każdym neuronem z poprzedniej warstwy. Te warstwy finalizują proces analizy danych i generują ostateczne wyniki, takie jak klasyfikacja obrazu. Po kilku blokach dane są przekształcane przez fully connected layer. W tej warstwie każdy neuron jest połączony z neuronem. Wyjściem tej warstwy są wyniki, czyli wektor prawdopodobieństw klas. [13]

2.2.1. Warstwa konwolucyjna

W warstwie konwolucyjnej przeprowadzana jest operacja splotu. Każdy z elementów macierzy wejściowej jest osobno przemnażany przez macierz zwaną jądrem (ang. kernel) lub filtrem (ang. filter), a następnie wyniki mnożenia są sumowane i przepisywane do macierzy wyjściowej. Oto przykład [13] wzoru dla operacji splotu dla macierzy dwuwymiarowej:

$$\text{Conv2d}(f, w)_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} f_{(i+m),(j+n)} \cdot w_{m,n}$$

Pierwsze warstwy konwolucyjne wyodrębniają generalne, niskopoziomowe cechy, jak na przykład linie i krawędzie, a następne warstwy uczą się bardziej szczegółowych cech.

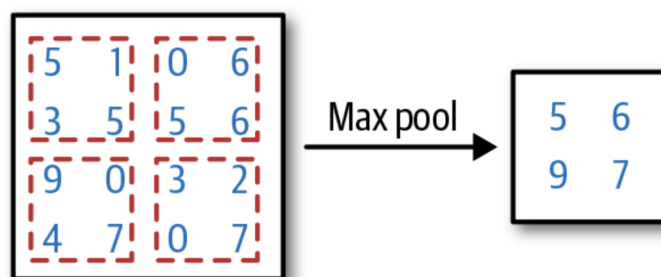


Rysunek 2.1: Przykład warstwy konwolucyjnej (źródło [13])

2.2.2. MaxPooling

Pooling ma za zadanie redukcji wymiarowości danych przez stopniowe zmniejszenie obrazu, co przekłada się na zmniejszenie parametrów do wytrenowania. Warstwa ta przyspiesza to trening modeli i zwiększa odporność na szum. W programie został wykorzystany Max Pooling czy

Operacja Max Pooling polega na podziale wejściowego obrazu na tabele i zastąpieniu każdej tabeli maksymalną wartością. Zazwyczaj okno ma rozmiar 2x2 lub 3x3, a krok przesunięcia (stride) wynosi 2, co oznacza, że tabela jest przesuwana o 2 piksele wzdłuż obrazu.



Rysunek 2.2: Max Pooling (źródło [13])

2.2.3. Funkcje aktywacji

Funkcje aktywacji odpowiedzialne są za wprowadzenie nieliniowości do modelu. Są one ważne, aby wprowadzić nieliniowe relacje pomiędzy danymi wejściowymi tak aby wynik nie był zapisany jako liniowa kombinacja danych wejściowych.

Relu

Funkcja Relu polega na eliminacji negatywnych wartości, która jest łatwą operacją matematyczną przez to jest relatywnie efektywna przy trenowaniu modelu. Pomaga w zwalczaniu znikającego gradientu.

$$f(x) = \begin{cases} 0, & \text{jeśli } x < 0 \\ x, & \text{jeśli } x \geq 0 \end{cases}$$

x oznacza wektor wartości wejściowych.

Softmax

Funkcja softmax jest używana do modelowania problemów klasyfikacji wielu klas, gdzie celem jest uzyskania wyjścia w postaci rozkładu prawdopodobieństwa dla różnych możliwych klas [11].

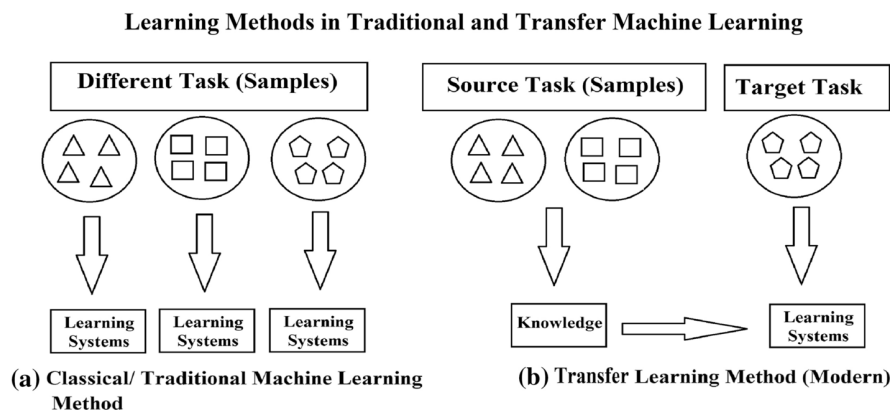
$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

W powyższej formule:

- x_i oznacza wektor wartości wejściowych dla i-tego elementu,
- x_j oznacza wektor wartości wyjściowych dla j-tego elementu,
- e^{x_i} oznacza standardową funkcję wykładniczą dla wektora wejściowego,
- N oznacza całkowitą liczbę elementów wejściowych.

2.3. Transfer learning

Transfer learning wykorzystuje nabytą wiedzę z wcześniejszych zadań do zrealizowania docelowego zadania [8]. W przeciwieństwie do tradycyjnych technik uczenia maszynowego, gdzie jeden model jest trenowany na jednym zbiorze do wykonania pojedynczego zadania (rys 2.1).



Rysunek 2.3: Traditional/Classical ML vs. TL (źródło [9])

Dzięki TL możliwe jest skuteczne wykorzystanie istniejących zasobów i ograniczenie potrzebnych zasobów obliczeniowych. TL może być wykorzystywany przy ograniczonej ilości danych, dzięki nabytej wiedzy z rozpoznawania ogólnych cech i wzorców. Szczególnie w przypadku ograniczonej liczbie danych medycznych, wstępnie nauczony model, może zostać dostosowany do rozpoznawania konkretnych patologii lub cech medycznych w skanach.

Najbardziej trafne modele, to takie które zostały wytrenowane na bardzo dużym zbiorze danych (np. ImageNet). Posiadają one kilka/kilkanaście warstw konwolucyjnych i warstw w pełni połączonych, które są w stanie wykrywać różne cechy obrazów na różnych poziomach abstrakcji. W mojej pracy wykorzystane zostały modele tj. VGG16, VGG19 i Inceptionv3, które zostały opisane szczegółowiej w dalszych podrozdziałach.

2.4. Model VGG16

VGG16 jest modelem opartym na konwolucyjnej sieci neuronowej stworzonej przez K. Simonyan and A. Zisserman [10]. Model został wytrenowany na bardzo dużym zbiorze danych ImageNet składającego się z ponad 14 milionów kolorowych (RGB) zdjęć. Składa się z następujących warstw (rys 2.2).

Wejście to obraz RGB 224x224, czyli macierz 224x224x3. Każdy blok składa się z kilku konwolucyjnych warstw z jądrem 3x3 z podaną na (rys 2.2) ilością filtrów. Domyślnym wyjściem modelu jest wektor, składający się z 1000 wartości.

$$\mathbf{y} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{999} \end{bmatrix}$$

Każda wartość wektora reprezentuje prawdopodobieństwo klasyfikacji obrazu do poszczególnej klasy.

2.4.1. Zastosowanie TL

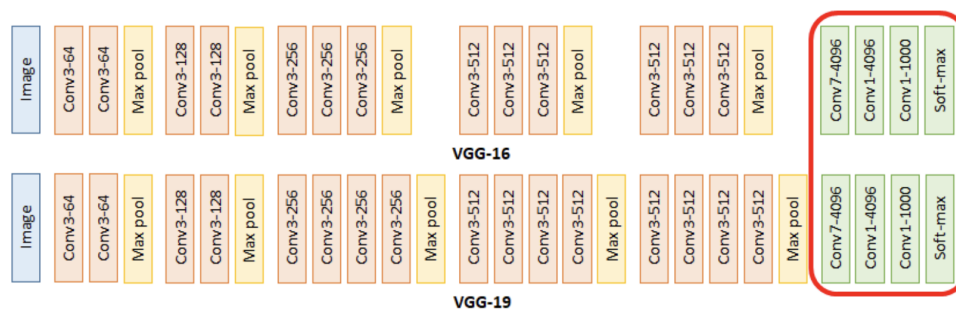
Według zasad TL do klasyfikacji obrazu bloki 1-5 zostają zamrożone. Blok 6 zostaje zastąpiony przez klasyfikator, dostosowany do docelowego zadania - klasyfikowania nowotworu mózgu. Ostatnia warstwa zwraca wektor zawierający 3 klasy.

2.5. Model VGG19

Główną różnicą pomiędzy VGG19 i VGG16, jest dodanie 3 dodatkowych warstw konwolucyjnych. Każda dodatkowa warstwa jest analogiczna do tej znajdującej się już w danym bloku. Warstwy zostały dodane w bloku trzecim, czwartym i piątym. Dodatkowe warstwy mają za zadanie zwiększyć możliwość rozpoznawania bardziej zaawansowanych wzorców. VGG19 jest bardziej kosztowny obliczeniowo.

Zastosowanie TL

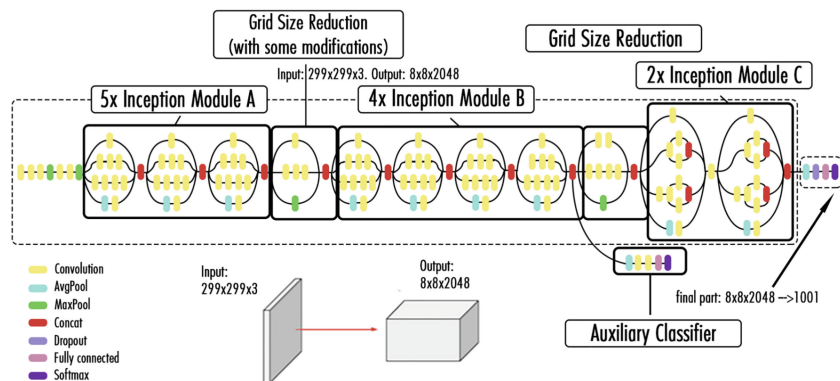
TL został zastosowany analogicznie jak w modelu VGG16 z zmienionymi parametrami.



Rysunek 2.4: Architektura VGG16 i VGG19 (źródło [17])

2.6. Model Inceptionv3

Model Inceptionv3 został wytrenowany na bazie zdjęć. Architektura Inceptionv3 opiera się na wykorzystaniu modułów Inception. Każdy moduł oblicza równoległe oddzielną warstwę konwolucyjną (lub Max Pooling), która ma różny rozmiar filtra(kernel). Zastosowane zostały filtry 1x1, 3x3 i 5x5. Później wyniki z modułu są łączone aby wyodrębnić cechy. Dzięki temu zabiegowi model może uwzględnić cechy lokalne i globalne. [15] TL został zastosowany w ten sam sposób co poprzednio, czyli została zastąpiona ostatni blok z zmienionymi parametrami.



Rysunek 2.5: Architektura modelu Inceptionv3 (źródło [15])

Rozdział 3

Implementacja

3.1. Architektura programu

Zbiór danych - zdjęć skanu rezonansu magnetycznego znajdującego się w plikach programu został wcześniej obrobiony za pomocą biblioteki cv2. Oto następujące filtry, które zostały nasienione na każde użyte zdjęcie:

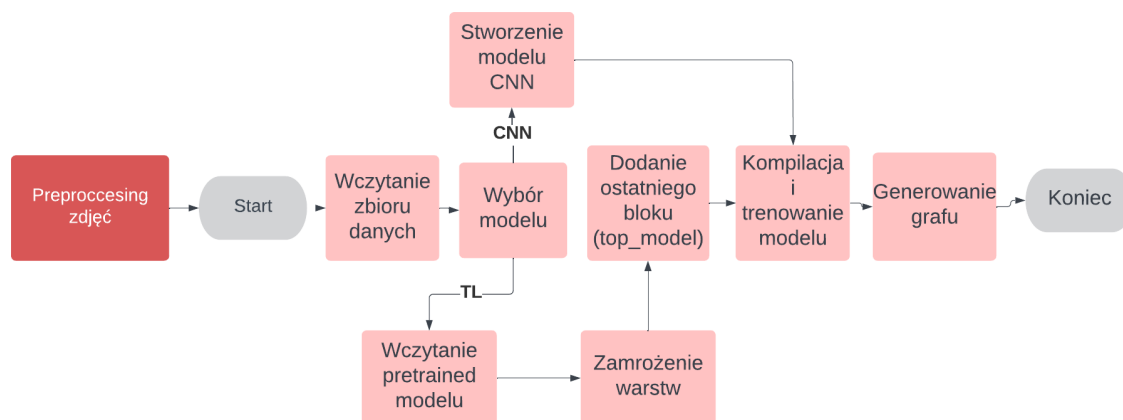
- rozmycie Gaussa w celu usunięcia szumu (cv2.GaussianBlur) [18]
- obcięcie niepotrzebnego obszaru
- progowanie (bineryzacja) obrazu (cv2.threshold)[19]

Przy starcie programu mamy wybór, który model (VGG16, VGG19, Inceptionv3) ma zostać wytrenowany. Wykonywane kroki podczas trenowania modelu przy użyciu TL:

1. Wczytanie wstępnie nauczonych wag z modelu nauczonego na dużym zbiorze danych (ImageNet)
2. Zamrożenie wszystkich wag w niższych warstwach (konwolucyjnych)
3. Zastąpienie górnej warstwy (top layer) sieci dostosowanym klasyfikatorem do docelowego zadania. W ostatniej warstwie liczba wyników musi być taka sama jak liczba klas.
4. Trenowanie jedynie dostosowanego klasyfikatora, dzięki temu optymalizujemy model (mamy mniej parametrów do wytrenowania).
5. Generowanie wykresów i metryk jakości modelu

Wykonywane kroki podczas trenowaniu modelu CNN:

1. Stworzenie modelu zawierający bloki, składające się z warstw konwolucyjnych z funkcjami aktywacji, Pooling layer, Flatten Layer
2. Kompilacja modelu CNN
3. Trenowanie modelu CNN
4. Generowanie wykresów i metryk jakości modelu



Rysunek 3.1: Architektura programu

3.2. Wymagania systemowe

Do uruchomienia programu należy zainstalować Python 3.10 i następujące biblioteki:

- tensorflow 2.12
- cv2 4.7.0
- keras 2.12.0
- sklearn 1.2.2
- tqdm 4.65.0
- numpy 1.23.5
- matplotlib 3.7.1

Program był testowany na komputerze firmy HP o procesorze Intel(R) Core(TM) i7-5500U CPU @ 2.40GHz, 8 GB pamięci RAM i 64-bitowym systemem operacyjnym (Windows 10). Trenowanie modeli pochłaniało wiele czasu i pamięci RAM, dlatego nastąpiła zmiana środowiska na Google Colab. Wirtualne środowisko korzystało z NVIDIA V100 Tensor Core, 16 GB RAM i 16 GB VRAM.

3.3. Opis klas i głównych metod

W plikach programu znajdują się zbiór danych, gdzie skany zostały poddane wstępnym przetwarzaniu (preprocessing).

```

1 def crop_unnecessary_space(image):
2     image = cv2.imread(image)
3     bimage = cv2.GaussianBlur(image, (3, 3), 0)
4     thresh = cv2.threshold(bimage, 70, 255, 0)[1]
5
6     contours, hierarchy = cv2.findContours(thresh, cv2.RETR_TREE, cv2.
7     CHAIN_APPROX_SIMPLE)
8     # Znalezienie największego konturu
9     c = max(contours, key=cv2.contourArea)
  
```

```

9     # Usuniecie niepotrzebnego konturu
10    x, y, w, h = cv2.boundingRect(c)
11    cropped = image[y:y + h, x:x + w]
12    # Normalizacja zdjęcia do odpowiedniego rozmiaru wejścia dla modelu
13    transformed_image = cv2.resize(cropped, dsize=(IMAGE_HEIGHT, IMAGE_WIDTH)
14    )
15    return transformed_image

```

main Moduł odpowiadający za wczytanie plików do treningu i testowania modelu. Odpowiedzialny za komunikację z użytkownikiem i wywołujący moduły modeli.

my_cnn Odpowiada za tworzenie, kompilację i trening/fitting modelu CNN.

```

1 model = Sequential()
2 model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape
   =(224, 224, 3)))
3 model.add(Conv2D(32, kernel_size=(3, 3), activation='relu'))
4 model.add(MaxPool2D(pool_size=(2, 2), strides=(2, 2)))
5 model.add(Dropout(0.3))
6
7 model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
8 model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
9 model.add(MaxPool2D(pool_size=(2, 2), strides=(2, 2)))
10 model.add(Dropout(0.3))
11
12 model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
13 model.add(MaxPool2D(pool_size=(2, 2), strides=(2, 2)))
14 model.add(Dropout(0.4))
15
16 model.add(Flatten())
17 model.add(Dense(128, activation='relu'))
18 model.add(Dropout(0.3))
19 model.add(Dense(3, activation='softmax'))
20
21 model.compile(optimizer='adam',
22               loss='categorical_crossentropy',
23               metrics=['accuracy'])
24
25 model.summary()
26
27 history = model.fit(data["x_train"], data["y_train"],
28                     epochs=35,
29                     verbose=1,
30                     validation_data=(data["x_test"], data["y_test"]))
31

```

accuracy_graph Generuje wykres dokładności i wykres straty dla k liczby epok.

confusion_matrix Generuje macierz pomyłek i znormalizowaną macierz pomyłek dla danych testowych.

Funkcje odpowiadające za modele korzystające z TL. W każdej z nich znajdują się wewnętrzna funkcja `top_model`, która definiuje ostatnią dodatkową warstwę i funkcja startująca model. Przykładem struktury modułu jest funkcja wywołująca model VGG16.

vgg16_tf

```
1 def top_of_model(bottom_model, num_classes):
2     top_model = bottom_model.output
3     top_model = Flatten()(top_model)
4     top_model = Dense(1024, activation='relu', kernel_regularizer=
5         regularizers.l2(0.0005))(top_model)
6     top_model = Dense(1024, activation='relu', kernel_regularizer=
7         regularizers.l2(0.0005))(top_model)
8     top_model = Dense(512, activation='relu', kernel_regularizer=regularizers
9         .l2(0.0005))(top_model)
10    top_model = tf.keras.layers.Dropout(rate=0.4)(top_model)
11    top_model = Dense(num_classes, activation='softmax')(top_model)
12    return top_model
13
14 def tf_model_vgg16(data):
15     vgg = vgg16.VGG16(weights='imagenet',
16         include_top=False,
17         input_shape=(224, 224, 3))
18
19     for layer in vgg.layers:
20         layer.trainable = False
21
22     FC_Head = top_of_model(vgg, 3)
23     model = Model(inputs=vgg.input, outputs=FC_Head)
24
25     model.compile(optimizer='adam', loss='categorical_crossentropy', metrics
26         =['accuracy'])
27
28     r_vgg16 = model.fit(data["x_train"], data["y_train"],
29         epochs=35,
30         validation_data=(data["x_test"], data["y_test"]),
31         verbose=1,
32         initial_epoch=0
33     )
34
35     #Generowanie wykres w i tabel
36     y_prediction = model.predict(data['x_test'])
37     y_prediction = np.argmax(y_prediction, axis=1)
38     y_test = np.argmax(data['y_test'], axis=1)
39
40     show_confusion_matrix(y_prediction, y_test, "Macierz b Ć Źd w VGG16 ")
41
42     tumor_table = ["oponiak", 'glejak', 'guz przysadki']
43     print(classification_report(y_test, y_prediction, target_names=
44         tumor_table))
45
46     return r_vgg16
```

vgg19_tf

```
1 def top_of_model(bottom_model, num_classes):
2
3     top_model = bottom_model.output
4     top_model = Flatten()(top_model)
5     top_model = Dense(1024, activation='relu', kernel_regularizer=
6         regularizers.l2(0.0005))(top_model)
7     top_model = Dense(1024, activation='relu', kernel_regularizer=
8         regularizers.l2(0.0005))(top_model)
```

```

7   top_model = Dense(512, activation='relu', kernel_regularizer=regularizers
    .l2(0.0005))(top_model)
8   top_model = tf.keras.layers.Dropout(rate=0.4)(top_model)
9   top_model = Dense(num_classes, activation='softmax')(top_model)
10  return top_model

```

inception_tf

```

1  def top_of_model(bottom_model, num_classes):
2
3      top_model = bottom_model.output
4      top_model = Flatten()(top_model)
5      top_model = Dense(1024, activation='relu', kernel_regularizer=
    regularizers.l2(0.0005))(top_model)
6      top_model = Dense(1024, activation='relu', kernel_regularizer=
    regularizers.l2(0.0005))(top_model)
7      top_model = Dense(512, activation='relu', kernel_regularizer=regularizers
    .l2(0.0005))(top_model)
8      top_model = tf.keras.layers.Dropout(rate=0.4)(top_model)
9      top_model = Dense(num_classes, activation='softmax')(top_model)
10     return top_model

```

3.4. Przykłady uruchomienia

Przed uruchomieniem należy zainstalować potrzebne biblioteki zawarte w wymaganiach systemowych. Program zawiera interfejs tekstowy. W terminalu należy przejść do folderu, w którym znajduje się plik i wykonać komendę: **python -m final**

Funkcjonalności programu:

- trenowanie wybranych modeli na podstawie bazy danych CE-MRI
- generowanie macierzy pomyłek
- generowanie wykresów funkcji straty i dokładności

Rozdział 4

Metodologia badań

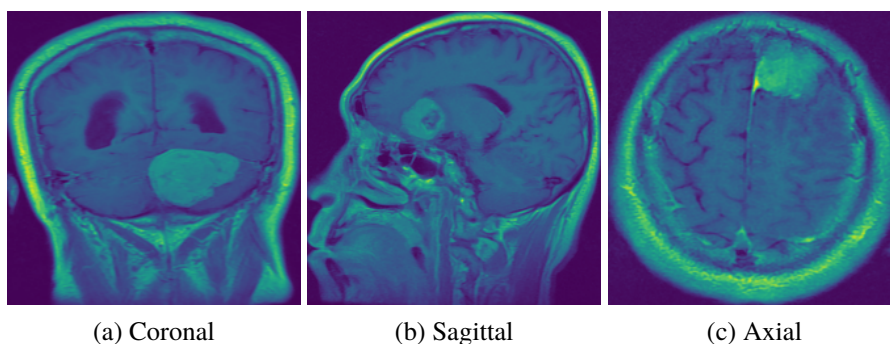
4.1. Opis zbioru danych - CE-MRI

Oryginalny zbiór danych CE-MRI jest publicznie dostępny [7]. Skany rezonansu magnetycznego zostały wykonane w latach 2005-2010 w szpitalu Nanfang, w Guangzhou w Chinach i w szpitalu uniwersytetu medycznego Tianjin w Chinach. W zbiorze występują trzy typy nowotworów: glejak, oponiak i guz przysadki. Każdy skan został poddany klasyfikacji przez ekspertów. Zbiór liczy dane 233 pacjentów z sumą 3064 zdjęć. Odpowiednio: Skany zostały wykonane pod różnym kątem (axial, coronal, sagittal), przykłady widoczne na rysunku 4.1. Każdy skan ma rozmiar 512 x 512 pikseli i jest czarno-biały. W pracy został wykorzystany zbiór przekonwertowanych skanów z rozszerzenia .mat do .png oraz zmianę kanału obrazu na kanał RGB.

Szczegóły zbioru danych CE-MRI		
Typ nowotworu	Liczba pacjentów	Liczba skanów
oponiak	82	708
glejak	89	1426
guz przysadki	62	930

Tabela 4.1: Szczegóły zbioru danych CE-MRI

Rysunek 4.1: Przykładowe skany rezonansu magnetycznego oponiaka



4.2. Miary jakości klasyfikacji

Analiza jakości klasyfikacji i wydajności jest kluczowa przy wykorzystywaniu modeli uczenia maszynowego. Poprzez badanie wyników i wyciąganie odpowiednich wniosków, możliwe jest ocenienie wpływu różnych parametrów na skuteczność algorytmu oraz porównanie go z innymi rozwiązaniami. Taka ocena umożliwia lepsze zrozumienie i optymalizację działania modelu, co prowadzi do doskonalenia efektywności klasyfikacji. W tym rozdziale przedstawione zostały miary używane przy analizie związanych z trenowaniem i testowaniem modeli.

Stosowane jest następujące nazewnictwo:

- TP (ang. True Positive) – liczba poprawnie sklasyfikowanych próbek danej klasy
- FN (ang. False Negative) – liczba błędnie sklasyfikowanych próbek danej klasy
- TN (ang. True Negative) – liczba próbek poprawnie nie przydzielonych do danej klasy
- FP (ang. False Positive) – liczba próbek błędnie przydzielonych do danej klasy

Dokładność (ang. accuracy)

$$\text{dokładność} = \frac{TP + TN}{TP + FP + FN + TN}$$

Precyzja (ang. precision) Określa jaka część wyników wskazanych przez klasyfikator jako dodatnie jest faktycznie dodatnia

$$\text{precyzja} = \frac{TP}{TP + FP}$$

Czułość (ang. recall) Określa jaką część dodatnich wyników wykrył klasyfikator

$$\text{czułość} = \frac{TP}{TP + FN}$$

Miara F Miara F może być interpretowana jako średnia ważona harmoniczna precyzji i czułości.

$$F_{\beta} = \frac{(1 + \beta^2) * \text{precision} * \text{recall}}{\beta^2 * \text{precision} + \text{recall}}$$

W miarze jakości F1, wartość β przyjmuje 1, dlatego określany jest jako F1-score (albo F-measure).

Macierz pomyłek Macierz pomyłek (ang. confusion matrix) jest prezentowana w postaci tabeli, gdzie wiersze odpowiadają oczekiwanym wynikom klasyfikacji, a kolumny przedstawiają klasy przypisane przez model. Tabela ta zawiera informację o liczbie poprawnie i błędnie przypisanych klas. Poniżej przedstawiona została standardowa macierz pomyłek, gdzie predicted class to klasy sklasyfikowane przez model, a actual class to rzeczywiste klasy.

		True/Actual Class	
		Positive (P)	Negative (N)
Predicted Class	True (T)	True Positive (TP)	False Positive (FP)
	False (F)	False Negative (FN)	True Negative (TN)
		$P = TP + FN$	$N = FP + TN$

Rysunek 4.2: Standardowa macierz pomyłek (źródło [12])

Jako, że w pracy zostają przewidywane 3 klasy, oto przykład macierzy pomyłek dla 3 klas.

		True Class		
		A	B	C
Predicted Class	A	TP_A	E_{BA}	E_{CA}
	B	E_{AB}	TP_B	E_{CB}
	C	E_{AC}	E_{BC}	TP_C

Rysunek 4.3: Macierz pomyłek dla 3 klas (źródło [12])

W powyższej tabeli:

- TP_A, TP_B, TP_C oznacza liczbę poprawnie sklasyfikowanych klas (TP)
- E_{AB} i reszta analogicznie oznacza liczbę błędów, czyli liczbę próbek, które należą do klasy A, ale zostały sklasyfikowane jako klasa B, więc zostały błędnie sklasyfikowane. Chcąc obliczyć standardową FN_A wystarczy $FN_A = E_{AB} + E_{AC}$.

Znormalizowana macierz pomyłek jest modyfikacją standardowej macierzy pomyłek, w której wartości są przeskalowane w celu uzyskania rozkładu prawdopodobieństwa. Zamiast absolutnych liczb przypadków, dzięki normalizacji przedstawiane są stosunki procentowe lub prawdopodobieństwa przypisań klas.

Rozdział 5

Wyniki

5.1. Wyniki badań

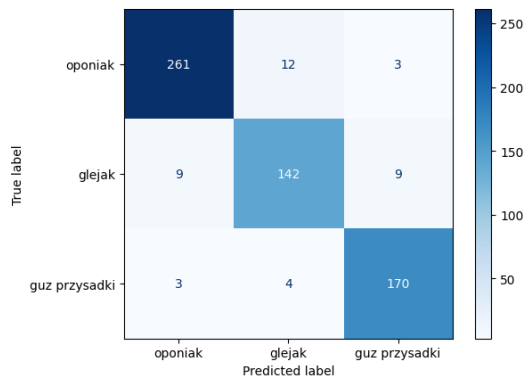
5.1.1. Wyniki VGG16

Tabela 5.1 prezentuje wynik klasyfikacji. Najlepiej została przewidziany oponiak. Z analizy zbioru walidacyjnego na Rysunku 5.2 wynika, że trening można przerwać po 10 epokach.

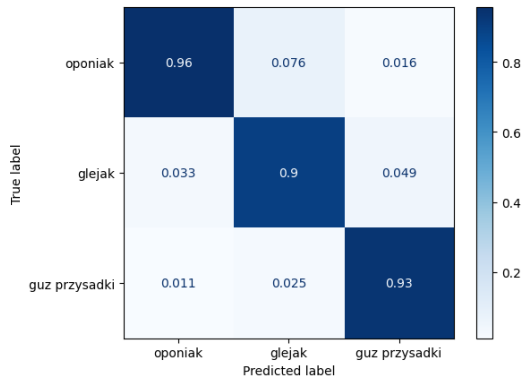
Tabela jakości klasyfikacji dla VGG16				
Klasa	Precyzja	Czułość	F1	Support
oponiak	0.96	0.95	0.95	276
glejak	0.90	0.89	0.89	160
guz przysadki	0.93	0.96	0.95	177

Tabela 5.1: Tabela jakości klasyfikacji dla VGG16

Dokładność na zbiorze testowym: 0.93

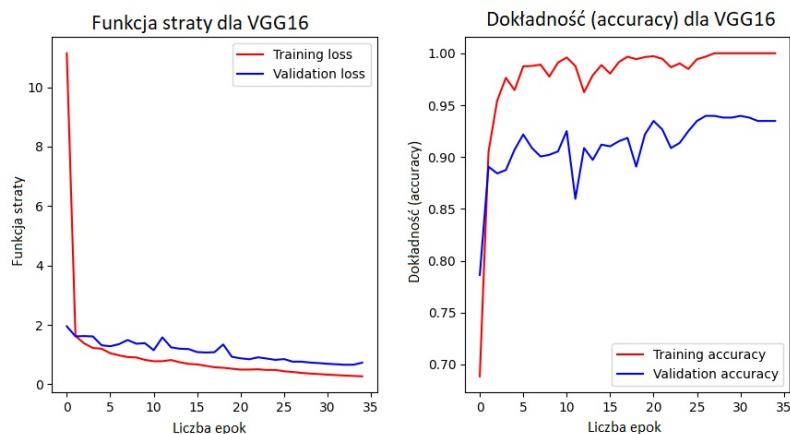


(a) Macierz pomyłek VGG16



(b) Znormalizowana macierz pomyłek VGG16

Rysunek 5.1: Macierze pomyłek VGG16



Rysunek 5.2: Funkcja strat i dokładność przy 35 epokach modelu VGG16

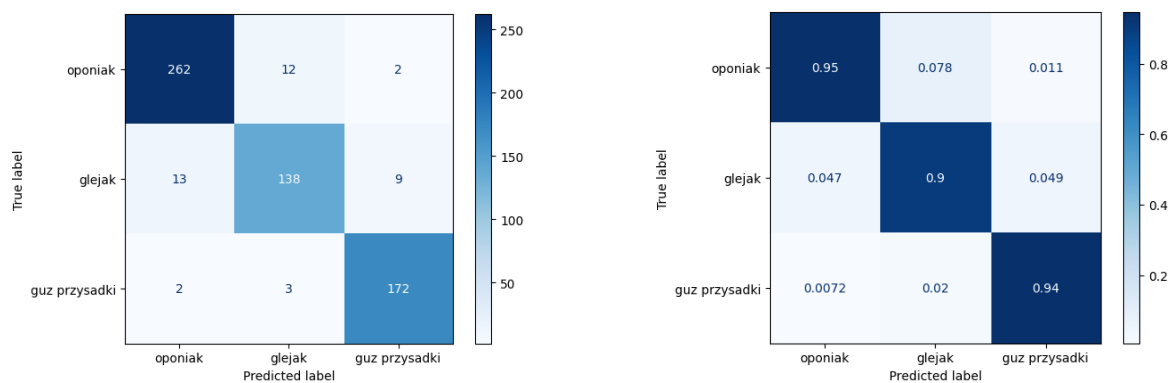
5.1.2. Wyniki VGG19

Tabela 5.2 przedstawia wynik klasyfikacji. Najlepiej została przewidziany oponiak. Ze zbioru walidacyjnego na Rysunku 5.3a wynika, że trening można przerwać po 9-10 epokach. Wyniki są bardzo podobne do modelu VGG16.

Tabela jakości klasyfikacji dla VGG19				
Klasa	Precyzja	Czułość	F1	Support
oponiak	0.95	0.95	0.95	276
glejak	0.90	0.86	0.88	160
guz przysadki	0.94	0.97	0.96	177

Tabela 5.2: Tabela jakości klasyfikacji dla VGG19

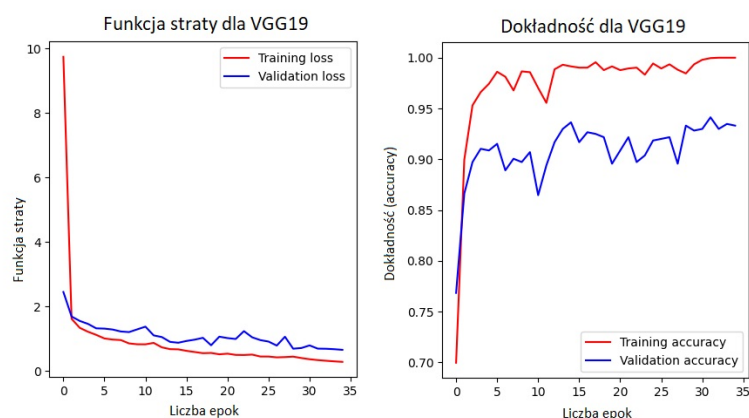
Dokładność na zbiorze testowym: 0.93



(a) Macierz pomyłek VGG19

(b) Znormalizowana macierz pomyłek VGG19

Rysunek 5.3: Macierze pomyłek VGG19



Rysunek 5.4: Funkcja strat i dokładność przy 35 epokach modelu VGG19

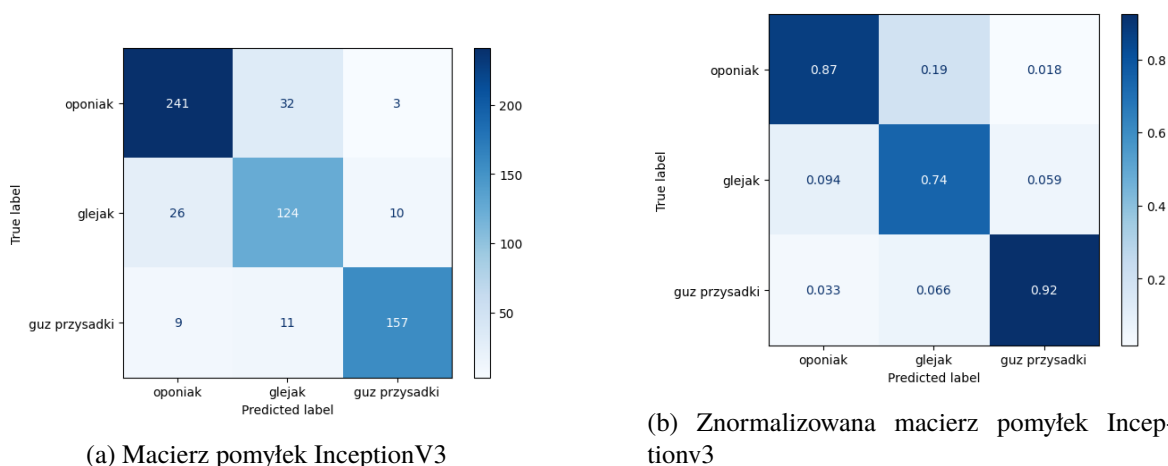
5.1.3. Wyniki Inceptionv3

Wyniki klasyfikacji są prezentowane w tabelce 5.3. Najlepiej została przewidziana guz przysadki. Z analizy zbioru walidacyjnego na Rysunku 5.5a wynika, model jest niestabilny i nie poprawia swojego wyniku względem epok. Funkcja straty jest bardzo wysoka, która waha się w zakresie 10-20.

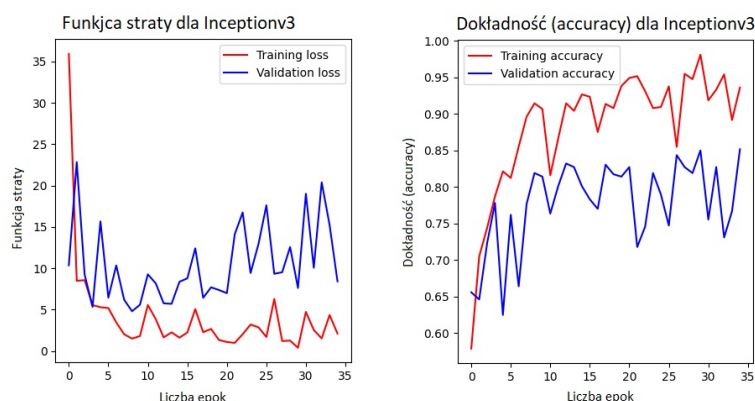
Tabela jakości klasyfikacji dla Inceptionv3				
Klasa	Precyzja	Czułość	F1	Support
oponiak	0.87	0.87	0.87	276
glejak	0.74	0.78	0.76	160
guz przysadki	0.92	0.89	0.90	177

Tabela 5.3: Tabela jakości klasyfikacji dla Inceptionv3

Dokładność na zbiorze testowym: 0.85



Rysunek 5.5: Macierze pomyłek Inceptionv3



Rysunek 5.6: Funkcja strat i dokładność przy 35 epokach modelu Inceptionv3

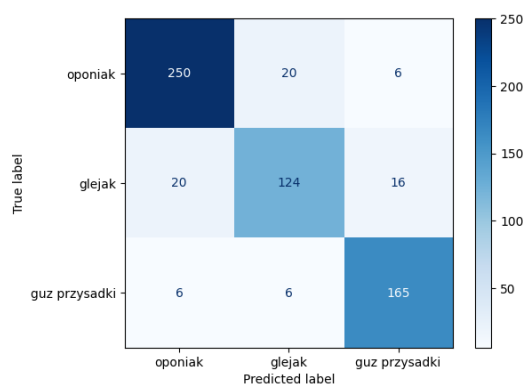
5.1.4. Wyniki własnej sieci CNN

W Tabelce 5.4 prezentowany jest wynik klasyfikacji. Najlepiej została przewidziany oponiak. Z analizy zbioru walidacyjnego na Rysunku 5.7a wynika, że trening można przerwać po 15 epokach.

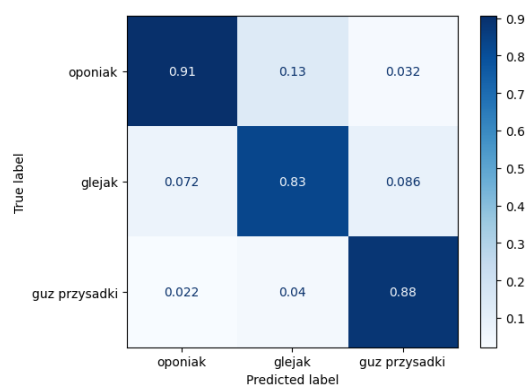
Tabela jakości klasyfikacji dla CNN				
Klasa	Precyzja	Czułość	F1	Support
oponiak	0.91	0.91	0.91	276
glejak	0.83	0.78	0.80	160
guz przysadki	0.88	0.93	0.91	177

Tabela 5.4: Tabela jakości klasyfikacji dla CNN

Dokładność na zbiorze testowym: 0.88

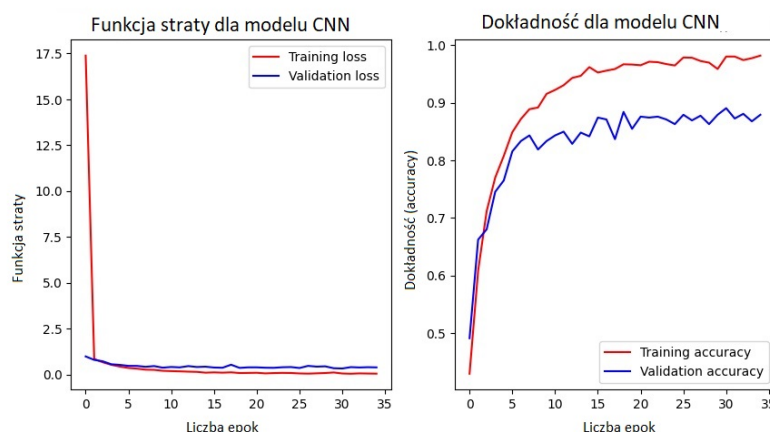


(a) Macierz pomyłek CNN



(b) Znormalizowana macierz pomyłek CNN

Rysunek 5.7: Macierze pomyłek CNN



Rysunek 5.8: Funkcja strat i dokładność przy 35 epokach modelu CNN

5.1.5. Porównanie czasu treningu modeli

W Tabeli 5.5 przedstawiono czas obliczeń. Celem eksperymentu jest wpływ czasu treningu sieci na dokładność. Najszybciej uczył się model VGG19, którego dokładność wynosiła 0.93.

Czas treningu i testowania poszczególnych modeli	
Model	Czas
VGG16	3min 1s
VGG19	3min 39s
Inceptionv3	2min 53s
CNN	2min 55s

Tabela 5.5: Czas treningu i testowania poszczególnych modeli

5.2. Analiza wyników

Z powyższych wyników wynika, że najlepszym modelem okazał się model VGG16 ze względu na najwyższą dokładność na zbiorze testowym i porównywalnym czasem do pozostałych modeli. Bardzo porównywalne wyniki pod kątem dokładności uzyskał model VGG19, jednak czas był znacząco dłuższy. Jest to przewidywalny wynik, jako że model VGG16 i VGG19 są bardzo podobne do siebie, w VGG19 wprowadzone są dodatkowe warstwy, które wydłużają trening. Oba te modele przetrenowują się około 10 epoki, co widać na wykresach dokładności gdzie występuje chwilowy spadek w grafie. Najgorsze wyniki uzyskał model Inceptionv3, gdzie patrząc na macierz pomyłek popełnia wiele błędów w klasyfikacji nowotworu, szczególnie w rozpoznaniu glejaka. Mimo wprowadzenia regularyzacji, patrząc na wykresy funkcji strat i dokładności są bardzo niestabilne. Podobny wynik tego modelu został uzyskany w następującym artykule [16], gdzie na wykresie dokładności i funkcji strat występuje wiele fluktuacji. Wyniki na własnej sieci CNN są satysfakcjonujące. Wykresy funkcji starty i dokładności zachowują się stabilnie. Funkcja straty na zbiorze testowym i treningowym są zbieżne ze sobą.

5.3. Dyskusja - porównanie wyników

Możemy wyciągnąć wniosek, że stworzona własna sieć CNN, która składa się z wielu warstw konwolucyjnych i została wytrenowana na odpowiednio dużej ilości danych jest porównywalnie dobra z gotowymi, wcześniej wytrenowanymi modelami o zaawansowanej architekturze, gdzie został zastosowany TL. Wyniki modelu Inceptionv3 znacząco odbiegają od pozostałych, co może świadczyć o złym doborze parametrów lub ze względu na charakter tego modelu (moduły Inception), nie jest on na tyle uniwersalny (nawet po zastosowaniu TL) do zadania klasyfikacji nowotworów.

Rozdział 6

Podsumowanie

W niniejszej pracy licencjackiej zostały omówione takie zagadnienia jak konwolucyjna sieć neuronowa, transfer learning (TL), modele VGG16, VGG19 i Inceptionv3. Celem pracy było zbadanie metod klasyfikacji typów nowotworu mózgu, objawiających się na skanach rezonansu magnetycznego z zbioru CE-MRI.

Udało się go zrealizować przez implementację VGG16, VGG19, Inceptionv3 modeli przy zastosowaniu TL i stworzoną sieć CNN przez napisany pakiet w języku Python. Może być on używany do badania skuteczności danych zastosowanych modeli. Implementacja została ona przedstawiona w rozdziale 3, a wyniki w rozdziale 5. Jednym z problemów przy implantacji był długi czas treningu modeli, który został rozwiązany przez przeniesienie testów do wirtualnego środowiska Google Collab.

Analizując wyniki, model VGG16 okazał się najskuteczniejszy, osiągając wysoką dokładność i porównywalny czas treningu z innymi modelami. Podobne wyniki osiągnął model VGG19. Model InceptionV3 uzyskał najgorsze wyniki, wykazując problemy w klasyfikacji nowotworów. Własna sieć CNN osiągnęła zadowalające rezultaty, porównywalne z gotowymi modelami o zaawansowanej architekturze, potwierdzając jej skuteczność po zaimplementowaniu odpowiednio zaawansowanych bloków i wytrenowaniu na odpowiednio dużym zbiorze danych. Główny wniosek wyciągnięty z badania to, że przy odpowiednio dużym zbiorze danych modele z zaaplikowanym TL i sieć CNN stworzona pod konkretne zadania może dawać porównywalne wyniki.

Dalsze badanie transfer learning i sieci konwolucyjnych w kontekście klasyfikacji nowotworów mózgu jest ważne, gdyż obecnie wymagana jest do tego specjalistyczna wiedza eksperta przez co postawienie diagnozy jest zależne od dostępności specjalistów. Następnym krokiem w rozwoju pracy mógłby być badania prowadzone na mniejszym zbiorze danych, gdzie modele z użyciem transfer learning powinny uzyskać lepsze wyniki od klasycznej sieci konwolucyjnej.

Bibliografia

- [1] Janus-Szymańska GM, Waszczuk Ł, Jacków-Nowicka J. *Significance of genetic and radiological examinations in diagnosis and therapy of brain glioma in adult patients.*, NOWOTWORY J Oncol 2021; 71: 328–334
- [2] Michael Hallett, Towards a theory of mathematical research programmes (I), *British Journal for the Philosophy of Science* vol. 30 (1979), pp. 1–25.
- [3] Juan Miguel Valverde, Vandad Imani, Ali Abdollahzadeh, Riccardo De Feo, Mithilesh Prakash, Robert Ciszek, Jussi Tohka *Transfer Learning in Magnetic Resonance Brain Imaging: a Systematic Review* J. Imaging 2021
- [4] Zar Nawab Khan Swati, Qinghua Zhao, Muhammad Kabir, Farman Ali, Zakir Ali, Saeed Ahmed, Jianfeng Lu, *Brain tumor classification for MR images using transfer learning and fine-tuning* Computerized Medical Imaging and Graphics, Volume 75, 2019, Pages 34–46, ISSN 0895-6111
- [5] Hu LS, Yoon H, Eschbacher JM, Baxter LC, Dueck AC, Nespodzany A, Smith KA, Nakaji P, Xu Y, Wang L, Karis JP, Hawkins-Daarud AJ, Singleton KW, Jackson PR, Anderies BJ, Bendok BR, Zimmerman RS, Quarles C, Porter-Umphrey AB, Mrugala MM, Sharma A, Hoxworth JM, Sattur MG, Sanai N, Koulemberis PE, Krishna C, Mitchell JR, Wu T, Tran NL, Swanson KR, Li J. *Accurate Patient-Specific Machine Learning Models of Glioblastoma Invasion Using Transfer Learning*. AJNR Am J Neuroradiol. 2019 Mar;40(3):418–425.
- [6] Gómez-Guzmán MA, Jiménez-Beristáin L, García-Guerrero EE, López-Bonilla OR, Tamayo-Perez UJ, Esqueda-Elizondo JJ, Palomino-Vizcaino K, Inzunza-González E. *Classifying Brain Tumors on Magnetic Resonance Imaging by Using Convolutional Neural Networks*. Electronics. 2023; 12(4):955.
- [7] Cheng J, Huang W, Cao S, Yang R, Yang W, *Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition*. Cheng, Jun (2017). *brain tumor dataset*. figshare. Dataset. PLOS ONE 10(12): e0144479.
- [8] Hosna, A., Merry, E., Gyalmo, J. *Transfer learning: a friendly introduction*. J Big Data 9, 102 (2022).
- [9] Pan SJ, Yang Q. A survey on transfer learning. *IEEE Trans Knowl Data Eng.* 2009;22(10):1345–59
- [10] K. Simonyan, and A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, CoRR, 2014
- [11] Lavika Goel, Akshina Jindal, Shray Mathur, *Chapter 4 - Design and implementation of a crop recommendation system using nature-inspired intelligence for Rajasthan, India*, In Cognitive Data

Science in Sustainable Computing, Deep Learning for Sustainable Agriculture, Academic Press, 2022, Pages 109-128

- [12] Tharwat, Alaa 2021, "Classification assessment methods", *Applied Computing and Informatics*, Vol. 17 No. 1, pp. 168-192
- [13] Seth Weidman *Deep Learning from Scratch* O'Reilly Media, Inc.
- [14] Srikanth Tammina *Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images*
- [15] Zorgui, Sana Chaabene, Siwar Bouaziz, Bassem Batatia, Hadj Chaari, Lotfi. (2020). *A Convolutional Neural Network for Lentigo Diagnosis*.
- [16] Srigiri Krishnapriya, Yepuganti Karuna *Pre-trained deep learning models for brain MRI image classification*
- [17] Ghada A. Shadeed, Mohammed A. Tawfeeq and Sawsan M. Mahmoud *Automatic Medical Images Segmentation Based on Deep Learning Networks IOP Conference Series: Materials Science and Engineering, Volume 870*
- [18] *Dokumentacja cv2- https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html*
- [19] *Dokumentacja cv2 https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html*